

SBツール

web-exp-tool - web実験ツール 環境構築手順書

目次

概要

web-exp-tool - web実験ツールの環境構築手順

firebaseのセットアップ、リポジトリのセットアップ

要件

* volta - node/npm version管理ツール

<https://docs.volta.sh/guide/getting-started>

* firebase-tools - firebase管理コマンドラインツール

volta インストール後に、グローバルでインストールする。

* <https://firebase.google.com/docs/cli?hl=ja#install-cli-mac-linux>

* vue-cli - vueアプリケーション管理コマンドラインツール

volta インストール後に、グローバルでインストールする。

* <https://cli.vuejs.org/#getting-started>

* gcloud/gsutil - google cloud, cloud storage管理用コマンドラインツール

* https://docs.cloud.google.com/storage/docs/gsutil_install?hl=ja

* <https://github.com/SandBoxInc/web-exp-tool>

* 事前にcloneしておく

firebaseのセットアップ

1. develop1@sandbox-vr.comでfirebaseコンソールにログインしてweb-exp-tool-dev, web-exp-tool-prodの2つのプロジェクトを生成する

2. プロジェクトの設定

作成した2つのプロジェクトに以下の設定を行う

* プロジェクト設定

* GCPリソースロケーションの選択

* マイアプリの作成

* アプリ名はプロジェクトと同じ名前を使用

* 「Firebase Hostingも設定します」にチェックを入れる

* authenticationサービスのセットアップ

* メール/パスワード プロバイダを選択する

* 「メール/パスワードを有効にする」のトグルを有効化して保存

* firestore databaseサービスのセットアップ

* 「本番環境モードで開始する」にチェックをいれて次に進み有効化する

* 以下のルールでルールを更新して公開する

```
rules_version = '2';
```

```
* service cloud.firestore {
```

```
* match /databases/{database}/documents {
```

```
* match /{document=**} {
```

```
* allow read, write: if request.auth != null;
```

```
* }
```

```
* }
```

```
* }
```

```
*
```

* storageサービスのセットアップ

* 以下のルールでルールを更新して公開する

```
rules_version = '2';

* service firebase.storage {
  * match /b/{bucket}/o {
    * match /{allPaths=**} {
      * allow read, write: if request.auth!=null;
    * }
  * }
* }
```

CORSの設定

リポジトリの直下で以下を実行し、認証を通す

```
gcloud auth login
```

cors.json を適用する

```
gcloud config set project <your-project-id> // firebaseコンソールから確認可能
```

```
gsutil cors set cors.json gs://web-exp-tool-dev.appspot.com/
```

```
gsutil cors set cors.json gs://web-exp-tool-prod.appspot.com/
```

リポジトリのセットアップ

リポジトリのダウンロード(すでにcloneしてある場合は省略)

```
git clone <https://github.com/SandBoxInc/web-exp-tool.git>
```

1 / 14

インストール

```
cd web-exp-tool && npm install
```

firebaseへのログイン

develop1@sandbox-vr.comのアカウントでログイン

```
firebase login
```

firebaseプロジェクトの初期化

```
firebase init
```

1. 以下にスペースでチェックを入れる

- * Firestore
- * Hosting: Configure files for Firebase Hosting and...
- * Storage
- * Realtime Database

2. Use an existing projectからweb-exp-tool-devを選択する。

3. ? What do you want to use as your public directory? (public)と聞かれるのでdistと入力

4. 他はすべてdefault(そのままEnter)でOK

環境の追加

同じアプリでweb-exp-tool-prodにもアクセスしたいのでプロジェクトを切り替えられるようにエイリアスを設定する

```
firebase use --add
```

以下でエイリアスを張る

- * web-exp-tool-dev->dev
- * web-exp-tool-prod->prod

.env ファイルの追加

1. 2つのプロジェクトの構成情報を取得する。

プロジェクトの設定 → 全般 → マイアプリ → SDKの設定と構成から config にチェックを入れオブジェクトをコピーし、それぞれメモ帳などに貼り付けておきます。

2. リポジトリ直下の .env.example を .env.development , .env.production としてコピーします。

`cp .env.example .env.development`

3. `cp .env.example .env.production`

4.

5. コピーした環境変数にそれぞれの構成情報をコピーします。

※この時 web-exp-tool-dev プロジェクトの構成情報は .env.development 、 web-exp-tool-prod プロジェクトの構成情報は .env.production となるよう、間違えないようコピーしてください。

アプリケーションの起動

基本的には以下のコマンドで開発者サーバーを起動します(開発環境のconfigを参照して開発環境のリソースにアクセスします)

```
npm run serve-dev
```

訳ありで本番環境に接続したい場合は

```
npm run serve-prod
```

コーディング

1. developからブランチを切り作業を行う

2. 作業が完了したら作業ブランチをプッシュしてPRを作成

3. reviewerがレビューを完了したらdevelopにマージされます

レビュー

reviewerは`git fetch origin pull/{PR番号}/head:PR-{PR番号}`などでPRをfetchしてローカルで動作を確認する 問題なければPRをマージしましょう

ビルド&デプロイ

新しい機能がdevelopにマージされたタイミングでビルド/デプロイを行います

以下のスクリプトがそれぞれの環境に対応したビルド/デプロイのコマンドになります

* dev

レビュー後PRがマージされたタイミングでGithub Actionが走り自動でデプロイされます

* prod

実験前の本番環境チェック

本番環境リリースPRがマージされたら

実験ツールの動作確認作業(ダブルチェック)

PRレビューの際ローカルで動作を確認しましたが最後にもう一度実験ツールの動作を確認します

開発環境でホスティングされている実験ツールにログインして動作を確認します

※本番環境は実際の実験でのみ使用します

=====

Web実験ツール開発ドキュメント

この資料の位置づけ

旧版・参考用：この資料には現在のリポジトリ構成と異なる設定方法が含まれています。環境構築や運用では、web実験ツール 環境構築手順書を優先してください。

Web実験ツール開発者ドキュメント

本ドキュメントには以下を記す

1. web実験ツールの構築

2. firebaseプロジェクトのセットアップ

3. ツールの開発フロー

※すでに実験ツールの構築が済んでいて新しく開発者として加わる場合は「2. firebaseプロジェクトのセットアップ」から参照してください

2 / 14

前提

必要なランタイム/パッケージ&ツール

- * node.js
- * firebase-tools(常に最新)
- * vue-cli

1. Web実験ツールの構築

プロジェクトの作成

develop1@sandbox-vr.comでfirebaseコンソールにログインしてweb-exp-tool-dev, web-exp-tool-prodの2つのプロジェクトを生成する

プロジェクトの設定

作成した2つのプロジェクトに以下の設定を行う

- * プロジェクト設定
 - * GCPリソースロケーションの選択
 - * マイアプリの作成
 - * アプリ名はプロジェクトと同じ名前を使用
 - * 「Firebase Hostingも設定します」にチェックを入れる
- * authenticationサービスのセットアップ
 - * メール/パスワード プロバイダを選択する
 - * 「メール/パスワードを有効にする」のトグルを有効化して保存
- * firestore databaseサービスのセットアップ
 - * 「本番環境モードで開始する」にチェックをいれて次に進み有効化する
 - * 以下のルールでルールを更新して公開する
- * rules_version = '2';
- * service cloud.firestore {
- * match /databases/{database}/documents {
- * match /{document=**} {
- * allow read, write: if request.auth != null;
- * }
- * }

```

* }
*
* storageサービスのセットアップ
* 以下のルールでルールを更新して公開する
* rules_version = '2';
* service cloud.firestore {
* match /databases/{database}/documents {
* match /{document=**} {
* allow read, write: if request.auth != null;
* }
* }
* }
*

```

CORSの設定

ここ参考にして認証を済ませておく

VSCodeのremote sshなどを使用している方は--console-onlyフラグが使えます

認証が終わったら以下を実行

```
gsutil cors set cors.json gs://web-exp-tool-dev.appspot.com/
```

```
gsutil cors set cors.json gs://web-exp-tool-prod.appspot.com/
```

2. firebaseプロジェクトのセットアップ

リポジトリのダウンロード

```
git clone <https://github.com/SandBoxInc/web-exp-tool.git>
```

インストール

```
cd web-exp-tool npm install
```

firebaseへのログイン

develop1@sandbox-vr.comのアカウントでログイン

```
firebase login
```

firebaseプロジェクトの初期化

Firestore, Hosting: Configure files for Firebase Hosting and..., Storageにスペースでチェックを入れる Use an existing projectからweb-exp-tool-devを選択する? What do you want to use as your public directory? (public)と聞かれるの dist/devと入力

他はすべてdefault(そのままEnter)でOK

```
firebase init
```

環境の追加

同じアプリでweb-exp-tool-prodにもアクセスしたいのでプロジェクトを切り替えられるようにエイリアスを設定する

```
firebase use --add
```

以下でエイリアスを張る

```
* web-exp-tool-dev->dev
```

```
* web-exp-tool-prod->prod
```

3 / 14

configファイルの保存

2つのプロジェクトから構成ファイルを取得する

プロジェクトの設定 -> マイアプリ -> SDKの設定と構成 -> 構成 にチェックを入れオブジェクトをコピーします src/config 配下にdevelopment.jsとproduction.jsを作成し以下の内容でそれぞれ保存します

※web-exp-tool-devから取得した構成オブジェクトはdevelopment.jsに、web-exp-tool-prodから取得した構成オブジェクトはproduction.jsに保存するように注意してください

```
// ↓ web-exp-tool-devからコピーした構成オブジェクトを貼り付けます
```

```
const firebaseConfig = {  
  apiKey: "AAAAAAAAAAAAAAAAAAAA",  
  authDomain: "BBBBBBBBBBBBBBBB",  
  projectId: "CCCCCCCCCCCCCCCC",  
  storageBucket: "DDDDDDDDDDDDDDDD",  
  messagingSenderId: "EEEEEEEEEEEEEE",  
  appId: "FFFFFFFFFFFFFFF",  
  measurementId: "GGGGGGGGGGGGGGG"  
};
```

```
export { firebaseConfig }
```

```
// ↓ web-exp-tool-prodからコピーした構成オブジェクトを貼り付けます
```

```
const firebaseConfig = {  
  apiKey: "AAAAAAAAAAAAAAAAAAAA",  
  authDomain: "BBBBBBBBBBBBBBBB",  
  projectId: "CCCCCCCCCCCCCCCC",  
  storageBucket: "DDDDDDDDDDDDDDDD",  
  messagingSenderId: "EEEEEEEEEEEEEE",  
  appId: "FFFFFFFFFFFFFFF",  
  measurementId: "GGGGGGGGGGGGGGG"  
};
```

```
export { firebaseConfig }
```

3. ツールの開発フロー

web-exp-toolのデフォルトブランチはdevelopでgit-flowにのっとり開発を行います

サーバーの起動

基本的には以下のコマンドで開発者サーバーを起動します(開発環境のconfigを参照して開発環境のリソースにアクセスします)

```
npm run serve-dev
```

訳ありで本番環境に接続したい場合は

```
npm run serve-prod
```

developが最新であるか確認

チーム開発

コーディング

1. developからブランチを切り作業を行う
2. 作業が完了したら作業ブランチをプッシュしてPRを作成
3. reviewerがレビューを完了したらdevelopにマージされます

レビュー

reviewerはgit fetch origin pull/{PR番号}/head:PR-{PR番号}などでPRをfetchしてローカルで動作を確認する 問題なければPRをマージしましょう

ビルド&デプロイ

新しい機能がdevelopにマージされたタイミングでビルド/デプロイを行います

以下のスクリプトがそれぞれの環境に対応したビルド/デプロイのコマンドになります

* dev

レビュー後PRがマージされたタイミングでGithub Actionが走り自動でデプロイされます

* prod

実験前の本番環境チェック

本番環境リリースPRがマージされたら

実験ツールの動作確認作業(ダブルチェック)

PRレビューの際ローカルで動作を確認しましたが最後にもう一度実験ツールの動作を確認します

開発環境でホスティングされている実験ツールにログインして動作を確認します

※本番環境は実際の実験でのみ使用します

=====

Emotiv脳波計測・データ同期 引継ぎ資料

この資料について

Emotivを利用した脳波計測について、ソフトウェアライセンス、生データの取得、実験ログとの時刻同期、計測後の保存作業をまとめた資料です。

使用するソフトウェアとライセンス

- * 計測には通常のEmotivProライセンスを使用します。
- * Cortex APIは使用していません。
- * 生データはEmotivProから手動でエクスポートします。

4 / 14

- * Raw EEG SDKは高額なため、導入を検討する場合は最新の料金と利用条件をEmotivへ確認してください。

計測からデータ保存までの流れ

1. EmotivProを使用して脳波を計測します。
2. 計測中または計測後に、EmotivPro上で生データを確認します。
3. 波形を確認し、解析から除外するリジェクトチャンネルを特定します。
4. EmotivProから生データを手動でエクスポートします。
5. エクスポートしたデータを所定のGoogle Driveへアップロードします。
6. リジェクトチャンネルの情報を、案件の解析担当者が確認できる場所へ記録します。

生データの確認、リジェクトチャンネルの特定、エクスポート、Driveへのアップロードを同じ作業の中で行うことで、計測後処理の重複を減らせます。

刺激・回答ログとの時刻同期

EmotivProと実験ツールは、いずれも計測用PCのローカル時刻を基準にタイムスタンプを記録します。

- * EmotivPro：脳波データの時刻をPCのタイムスタンプ基準で記録
- * 実験ツール：刺激提示時刻や被験者の回答時刻をローカルタイムスタンプで記録
- * 解析時：両方のタイムスタンプを使ってデータを後から突合

過去の検証では、おおむね±10ミリ秒以内の精度でした。ただし、Emotivのハードウェアからソフトウェアへ記録される際の時刻補正ロジックは公開情報だけでは確認できず、常に同じ精度が保証されるわけではありません。

計測前の確認

- * 計測用PCの日時とタイムゾーンが正しいこと
 - * EmotivProと実験ツールを同じPC時刻基準で動かしていること
 - * 実験ツールが刺激提示時刻と回答時刻を記録できること
 - * 保存先のDriveフォルダとファイル命名規則
 - * 計測終了後にリジェクトチャンネルを記録する場所
 - * 必要なEmotivProライセンスが有効であること
- 複数PCを使う場合は、計測前に端末間の時刻差を確認し、必要に応じて時刻同期を行ってください。

計測後の確認

- * 生データをエクスポートした
- * ファイルが正常に開ける
- * Driveへのアップロードが完了した
- * 刺激提示・回答ログが保存されている
- * リジェクトチャンネルが記録されている
- * 脳波データと実験ログをタイムスタンプで突合できる

注意事項

- * Cortex APIを前提としたリアルタイム取得構成ではありません。
- * 過去の±10ミリ秒という値は参考値であり、案件ごとに同期精度を確認してください。

- * PCの時刻変更、タイムゾーン変更、スリープ復帰、複数PC間の時刻差は同期ずれの原因になります。
- * 生データや被験者情報を含むファイルは、案件で定められたアクセス権と保存場所に従って管理してください。

情報源

- * Slack：Emotivのソフトウェアライセンスまわり
- * 確認日：2026年7月28日

=====

SandBoxInc リポジトリ一覧・概要

最終更新: 2026-07-28

対象: GitHub Organization SandBoxInc 配下の全 50 リポジトリ

1. この資料について

この資料は、SandBoxInc が保有するコード資産の全体像を把握するためのものである。

リポジトリ数が多く、共通基盤、案件用ツール、技術検証、旧資産が混在している。まず用途別の分類と重要度を確認し、必要なものだけ詳細を参照できる構成にしている。

各リポジトリについて、以下をまとめている。

- * 何を実装しているか
- * どの業務・研究・運用で使うか
- * 動かすために必要な環境
- * 基本的な起動・利用方法
- * 最近の更新状況
- * 関連リポジトリとの使い分け

内容は既定ブランチの README、依存関係ファイル、Docker 定義、主要な実装、コミット履歴をもとに記載した。README がないものや、雛形から更新されていないものは、分かる範囲だけを書いている。

2. 重要度の見方

A: 重要

現在の業務・実験・顧客案件・社内インフラに直接関係するもの、または複数プロジェクトで使う共通基盤。改修時は影響範囲を確認し、テストと復旧方法を用意する。

目安:

- * 2024 年以降にも機能更新がある
- * 共有ライブラリ、実験基盤、計測基盤、解析基盤である
- * 本番サイト、AWS、Firebase、データ管理などの運用に直結する
- * 複数の案件・プロトコル・API を含む

B: 重要な補助ツール

単独では中核ではないが、実験運用、アカウント作成、データ搬送、解析、可視化、開発環境などを支えるもの。

5 / 14

C: 限定用途・試作・旧資産

特定案件専用、技術検証、サンプル、テンプレート、または長期間更新されていないもの。

最終更新日は既定ブランチの履歴を基準にしている。別ブランチだけで開発している場合は、実際の活動時期とずれることがある。

3. 全体像

3.1 重要リポジトリの用途別分類

カテゴリー	リポジトリ	主な役割
実験・刺激提示・計測	web-exp-tool, fg001, fg002, time-sync-system	Web 実験、HuME 計測、刺激提示、複数端末の時刻同期
EEG・生体信号解析	eeganalyze, treeflow, fieldstudy-script	EEG 解析部品、解析パイプライン、案件別解析
実験データ運用	web-exp-db-manager, exp-account-manager, web-exp-uploader, trigger-splitter	Firestore データ搬送、実験アカウント、素材投入、データ分割
顧客・クラウド基盤	imagica-cloud-function, saliency-functions, milky-way	AWS Lambda/API/RDS、解析関数、CloudFormation
コーポレート基盤	sandbox-corporate-website, sandbox-corporate-proxy	WordPress 企業サイト、Traefik リバースプロキシ
実験オペレーション補助	shortcutkey-disabler	被験者 PC のキー・ショートカット制限
既存サービス	neume-front-end	Neume の Vue フロントエンド

3.2 全体を把握するときに最初に見るリポジトリ

全体を把握する場合は、次の順で確認すると分かりやすい。

1. web-exp-tool: Web 実験基盤
2. fg002: デスクトップ刺激提示基盤
3. fg001: HuME 計測・トリガーログ基盤
4. eeganalyze: EEG 解析の共通ライブラリ
5. treeflow: 案件横断の解析パイプライン
6. web-exp-db-manager / exp-account-manager: Web 実験の運用
7. imagica-cloud-function: AWS 上の顧客向け解析システム
8. sandbox-corporate-website / sandbox-corporate-proxy: 会社サイト運用

4. 重要リポジトリ

4.1 実験・刺激提示・計測

web-exp-tool

- * 重要度: A
- * 最終確認更新: 2026-04-09
- * 既定ブランチ: develop
- * 用途: Web ブラウザ上で実験、質問紙、認知課題などを実施する中核フロントエンド
- * 主な実装: Vue 2、Vue Router、Vuex、Vuetify、Firebase、Axios
- * 状況: PR 数と案件別更新が多く、2026 年にも質問紙文言や案件プロトコルの更新が続いている

前提環境:

- * Node.js と npm
- * Firebase の開発・本番プロジェクト設定
- * 案件ごとの環境変数
- * ブラウザ

基本操作:

```
npm install
```

```
npm run serve-dev
```

本番相当設定を使う場合:

```
npm run serve-prod
```

注意点:

- * README は Vue CLI の雛形に近く、実際の案件構成やデプロイ手順は十分に説明されていない。
- * Firebase の開発環境と本番環境を取り違えないこと。
- * 案件固有のプロトコルの変更は、質問文・ルーティング・ログ形式・既存データとの互換性をまとめて確認する。
- * 現在の実験運用を支える中心的なリポジトリの一つ。

fg002

- * 重要度: A
- * 最終確認更新: 2025-08-05 (ローカル作業コピーには 2026 年の追加仕様・変更資料も存在)
- * 既定ブランチ: master
- * 用途: Python 製の刺激提示デスクトップアプリ
- * 主な実装: Python、刺激コンポーネント、試行・プロトコル抽象化、イベントログ、IPC、PyInstaller
- * 対応例: Simon、Stroop、AX-CPT、VAS、Big Five、画像分類、誘導呼吸、質問紙

前提環境:

- * Windows を主対象とする Python 環境
- * プロジェクト定義に従った Python 依存関係
- * 画像・動画・音声などの刺激素材

* HuME 計測と連携する場合は fg001 側の計測アプリ

* 配布版作成時は PyInstaller

開発時の基本操作:

```
python -m stim_presenter
```

テスト:

```
pytest
```

運用の要点:

6 / 14

1. 使用するプロトコルを選択する。
2. HuME と併用する場合、計測側と同一のセッション名を設定する。
3. 刺激提示を開始し、終了後にイベント CSV とアプリログを確認する。
4. HuME 側の外部トリガーログと刺激提示側のイベントログが対応していることを確認する。

注意点:

- * 認知課題の正解キー、ブロック順、待機時間、練習条件は研究計画に直結するため、コードだけで変更しない。
- * docs/operation_manual_experiment_start.md などの運用資料を優先する。
- * プロトコル単位のテストが存在するため、変更後は関連テストを必ず実行する。

fg001

* 重要度: A

* 最終確認更新: 2025-08-13

* 既定ブランチ: master

* 用途: HuME デバイスの計測、リアルタイム波形表示、トリガー受信、CSV ログ保存

* 主な実装: Python、Tkinter、multiprocessing、シリアル通信、PyInstaller、ベンダー DLL

* 提供形態: CLI と GUI

主な構成:

- * humelib/: HuME デバイスと dongle の通信
- * measurement_cli/: CLI 操作
- * measurement_gui/: GUI、フィルタ、波形プロット
- * logger/: 独立プロセスでの CSV ロギング
- * external_trigger_sender.py: 外部プロセスからのトリガー送信
- * HuME_CLI.spec, HuME_GUI.spec: PyInstaller 定義

前提環境:

- * Windows
- * HuME Dongle / 受信機と計測デバイス
- * 対応する COM ポート
- * Python と Pipenv
- * 同梱 DLL が動作するアーキテクチャ

開発時の起動:

```
pipenv install
```

```
pipenv run python main.py
```

```
pipenv run python gui_main.py
```

運用の要点:

1. HuME Dongle を接続する。
2. GUI で COM ポートを選択し、デバイスをスキャンする。
3. 計測対象デバイスを選択する。
4. セッション名を設定して計測を開始する。
5. 刺激提示アプリと併用する場合、同じセッション名を使う。

6. 終了後、計測 CSV、トリガー CSV、アプリログを確認する。

注意点:

- * デバイス通信、ロガー、GUI が複数プロセスで動く。終了処理やキュー制御の変更はデータ欠損につながる。
- * 外部トリガーと生デバイストリガーの時刻精度が解析品質に直結する。
- * DLL と Pyinstaller 設定を変更する場合は、実機を使った配布版テストが必要。

time-sync-system

- * 重要度: A
- * 最終確認更新: 2026-01-20
- * 既定ブランチ: master
- * 用途: 複数 PC / クライアント間の時刻同期と同期ログ管理
- * 状況: README と標準マニフェストがなく、ログ形式・クライアント出力名の修正履歴が中心

想定される使用場面:

- * 複数端末で計測・刺激提示・収録を同時に行う実験
- * 後処理でイベント時刻を突き合わせる案件

利用前に確認すべきこと:

1. サーバー / クライアントの起動ファイル
2. 使用ポートとファイアウォール設定
3. 同期方式と時刻単位
4. 出力ログの形式
5. fg001、fg002、Web 実験ログとの対応方法

README がないため、現状では内容の把握に時間がかかる。起動コマンド、ネットワーク構成、ログ例を README に追加したい。

4.2 EEG・生体信号解析

eeganalyze

- * 重要度: A
- * 最終確認更新: 2023-12-29
- * 既定ブランチ: master
- * 用途: EEG を中心とする生体信号解析の共通 Python ライブラリ

7 / 14

- * Python: 3.9 以上
- * 主な依存: MNE、NumPy、pandas、SciPy、matplotlib、OpenCV、scikit-learn 系、openpyxl

主要モジュール:

- * データ入出力
- * 前処理、分離、時系列解析
- * 区間解析
- * モデル・統計処理
- * 案件別変換・解析スクリプト

インストール例:

```
pip install "eeganalyze @ git+ssh://git@github.com/SandBoxInc/eeganalyze.git"
```

利用方法:

- * 共通処理は eeganalyze / パッケージから import する。
- * 案件固有スクリプトは、入力フォルダ・電極構成・サンプリング周波数・マーカー定義を確認して実行する。
- * 新規案件では、既存案件スクリプトを直接改変するより、共通処理を再利用した新しいエントリーポイントを作る。

注意点:

- * リポジトリ直下に多数の歴史的・案件固有スクリプトがある。共通ライブラリと案件コードの境界が曖昧である。
- * Python と科学計算パッケージのバージョンが古めに固定されている。

* 新規案件は、可能なら treeflow または fieldstudy-script の構造を利用する。

treeflow

* 重要度: A

* 最終確認更新: 2025-04-01

* 既定ブランチ: master

* 用途: EEG・動画・グラフ処理などを、データノードと処理ツリーで再利用可能な解析パイプラインとして構成する

* 主な実装: Python、MNE、NumPy、pandas、SciPy、statsmodels、OpenCV、MoviePy、機械学習

設計上の目的:

* データの load/save と整形を DataNode で共通化する

* 大きなデータを常時メモリに保持せず、必要な時だけロードする

* 処理の一部実行やバージョン分けを、スクリプトの大幅変更なしで行う

* 案件固有の「処理間の整形」をプロジェクトスクリプト側に明示する

セットアップ:

```
pipenv install
```

```
pipenv run python script/template.py
```

実際には script/ 以下の案件別スクリプトを選んで実行する。

注意点:

* script/ に多数の顧客・PoC 名を含むファイルがある。共通パッケージ変更時は案件横断で影響を確認する。

* README の Usage は簡潔なので、対象スクリプトの入力パスと生成物をコードで確認してから実行する。

* eeganalyze と役割が重複する部分がある。新しい共通処理をどちらに置くか、先に責務を決める。

fieldstudy-script

* 重要度: B

* 最終確認更新: 2023-12-29

* 既定ブランチ: master

* 用途: 案件用解析スクリプト群

* Python: 3.9 以上

* 依存: Git、ffmpeg、Rye、eeganalyze

セットアップの概略:

```
rye sync
```

利用方法:

* script/ 内から対象案件の解析スクリプトを選ぶ。

* 入力データのフォルダ構成、ffmpeg の PATH、GitHub SSH 認証を準備する。

* eeganalyze を GitHub から取得できる必要がある。

位置づけ:

* 共通解析機能は eeganalyze

* パイプライン抽象化は treeflow

* 案件ごとの実行スクリプトは fieldstudy-script

4.3 Web 実験のデータ運用

web-exp-db-manager

* 重要度: A

* 最終確認更新: 2024-05-14

* 既定ブランチ: master

* 用途: 実験データを Firestore から取得し、Google Drive へアップロードする自動化 CLI

* 主な実装: Go、Cobra、Firebase/Firestore、Google Drive API

前提環境:

* 配布バイナリ、または Go 1.21 系

* Google Cloud サービスアカウント

* Google Drive API の権限

8 / 14

* ドメイン全体の委任が必要な運用では、管理者による設定

* WEB_EXP_DB_MANAGER_PATH などのアプリ用ディレクトリ設定

基本利用:

1. Releases から OS に対応するバイナリを取得する。
2. バイナリへ PATH を通す。
3. アプリ用ディレクトリと設定ファイルを配置する。
4. 対象環境・対象データを指定して CLI を実行する。
5. Firestore の取得結果と Drive のアップロード先を確認する。

注意点:

- * サービスアカウント鍵をリポジトリへコミットしない。
- * 本番 Firestore と開発 Firestore の対象を明示的に分ける。
- * Drive への重複アップロード、対象ユーザー、対象期間を実行前に確認する。

exp-account-manager

* 重要度: A

* 最終確認更新: 2024-08-23

* 既定ブランチ: master

* 用途: Web 実験ツールのアカウント作成、削除、利用制限

* 主な実装: Go、Firebase Admin、対話 CLI

前提環境:

- * OS 対応バイナリ、または Go
- * 開発・本番 Firebase プロジェクトそれぞれの管理用認証情報

利用方法:

1. Releases からバイナリを取得する。
2. 開発・本番の認証ファイルをローカルの安全な場所へ保存する。
3. README に従って環境変数へ絶対パスを設定する。
4. 操作対象環境を確認して、アカウント作成・削除・制限を実行する。

注意点:

- * 本番アカウントを変更する管理ツールであり、実行前の環境確認が必須。
- * 認証ファイルや利用者情報を文書・ログ・Git に含めない。

web-exp-uploader

* 重要度: B

* 最終確認更新: 2022-10-24

* 用途: web-exp-tool で使う画像・動画などの素材をアップロードする CLI

* 主な実装: Go 1.18、Firebase、Google Cloud Storage

基本的な使い方:

```
go mod tidy
```

```
go run .
```

実際のアップロード先、認証方式、引数はコードまたは Releases の CLI ヘルプで確認する。Web 実験の素材投入フローに残っている場合は B、別の運用へ移行済みなら C に下げられる。

trigger-splitter

* 重要度: B

* 最終確認更新: 2022-12-12

* 用途: Firestore の export JSON をドキュメント／ユーザー単位へ分割する

* 主な実装: Go 1.17、対話 CLI

セットアップ:

```
export TRIGGER_SPLITTER_ROOT="/path/to/trigger"
```

```
go mod tidy
```

```
go run main.go
```

Windows では同等の環境変数を設定する。UID 衝突回避処理が追加されているため、出力ファイル名やコレクション UID の仕様を下流処理と合わせる。

4.4 クラウド・顧客システム・社内インフラ

imagica-cloud-function

* 重要度: A

* 最終確認更新: 2024-04-18

* 既定ブランチ: master

* 用途: イマジカ向け解析クラウドの API、Lambda、RDS マイグレーション、S3 補助スクリプト

* 主な構成:

- * EEG、trigger、job、result API

- * EEG/trigger アップロードフック

- * 解析関数

- * RDS の users、eegs、triggers、jobs、results テーブル

* 主な技術: AWS CLI、AWS SSO、Docker、Python Lambda、RDS、S3、Cognito、golang-migrate

前提環境:

- * AWS CLI と SSO プロファイル

- * Docker

- * golang-migrate

- * 対象 AWS アカウントとリージョンへの権限

9 / 14

開始:

```
aws sso login --profile <profile>
```

その後、README の DB マイグレーション手順と、各 api/・function/ の Dockerfile に従う。

注意点:

- * DB マイグレーションは本番データへ影響する。対象 DB、方向、バックアップを確認する。

- * Cognito トークンや一時認証情報をログ・Git に残さない。

- * API と Lambda が共有する DB スキーマを変更する場合は、デプロイ順を設計する。

saliency-functions

* 重要度: B

* 最終確認更新: 2023-07-20

* 用途: 複数のサリエンシーモデルを AWS Lambda 用 Docker イメージとして管理

* 前提: Docker、pyenv、Pipenv、AWS ECR、Python 3.8 系

新規関数の基本フロー:

1. template ディレクトリを新しいモデル名でコピーする。

2. Pipenv で依存関係を準備する。

3. モデル単位で ECR リポジトリを作る。

4. Docker イメージをビルドして ECR へ push する。

5. Lambda のイメージを更新する。

注意点:

- * README に tmeplate という綴りがあり、実ディレクトリ名を確認する。

- * 各関数固有の設定は、関数ディレクトリ内 README を正とする。

milky-way

* 重要度: B

* 最終確認更新: 2023-05-29

* 用途: 社内アプリ・ツールを支える AWS インフラの CloudFormation

* 管理対象: VPC、Internet Gateway、Subnet、Route Table、EIP、NAT Gateway

使い方:

* CloudFormation テンプレートの差分を確認する。

* AWS の対象アカウント・リージョンを固定する。

* Change Set で変更内容を確認してから適用する。

注意点:

* インフラ変更の影響範囲が大きい。README から外部 Notion 文書も参照する。

* 自動適用ではなく、Change Set とロールバック方針を用意する。

sandbox-corporate-website

* 重要度: A

* 最終確認更新: 2025-11-05

* 既定ブランチ: develop

* 用途: 株式会社 SandBox の WordPress 企業サイト

* 状況: SEO、投稿除外、ショートコードなどの機能更新が継続

前提環境:

* WordPress

* VPS

* サイト固有のテーマ／プラグイン構成

* 開発環境・運用情報を記載した社内 Notion

* sandbox-corporate-proxy との関係確認

注意点:

* README は概要のみで、環境構築は社内文書に依存する。

* 本番反映前にバックアップ、ステージング確認、WordPress DB との互換性確認が必要。

sandbox-corporate-proxy

* 重要度: A

* 最終確認更新: 2025-04-04

* 用途: コーポレート系サービスの Traefik リバースプロキシ

* 前提: Docker Compose、Traefik、Basic 認証用 httpasswd

起動:

```
docker compose up -d
```

事前に config 配下へ WordPress 用と Traefik 用の httpasswd ファイルを作成する。

注意点:

* 認証ファイルを Git へ追加しない。

* ポート、TLS 証明書、DNS、WordPress コンテナとのネットワークを確認する。

* 構成が小さい一方、本番サイトの入口となるため重要度は高い。

4.5 実験オペレーション

shortcutkey-disabler

* 重要度: A

* 最終確認更新: 2025-12-13

* 用途: 被験者 PC で、実験中に使用できるキーとマウス操作を制限する

10 / 14

* 主な実装: AutoHotkey と配布 EXE

許可される主な入力:

- * A～Z
- * Enter、Backspace、矢印
- * F11
- * 左クリック、縦スクロール

制限できないと明記されている例:

- * Ctrl + Alt + Delete
- * Alt + F4
- * Ctrl + Shift + Esc

使い方:

1. 対応する EXE を実験 PC で起動する。
2. 許可キーと禁止キーを事前確認する。
3. 実験終了後にプロセスを終了し、通常入力へ戻ったことを確認する。

注意点:

- * OS のセキュリティキーを完全には禁止できない。
- * 実験プロトコルに必要なキーまで禁止していないか、実験前に通し確認する。

4.6 既存サービス

neume-front-end

- * 重要度: B
- * 最終確認更新: 2023-08-08
- * 既定ブランチ: develop
- * 用途: Neume の Web フロントエンド
- * 主な実装: Vue 2、Vue Router、Vuex、Vuetify、AWS Amplify、PDF.js

起動:

```
npm install
```

```
npm run serve
```

ビルド:

```
npm run build
```

注意点:

- * Vue 2 と古い Amplify 系依存が中心で、更新時には互換性確認が必要。
- * 現在のサービス運用状況は要確認。ひとまず優先度 B としている。

5. 補助・限定用途・旧資産

以下は、用途が限定される、試作である、README が不足している、または長期間大きな更新がないリポジトリである。

リポジトリ 区分 内容・環境・使い方 最終確認更新

go-cognito-account-creator B AWS Cognito にユーザーを作成し検証済みにする Go CLI。AWS SSO、リージョン、User Pool、メール、パスワードを指定して create を実行する。 2024-05-19

neume-account-creator B Neume のアカウントとルートフォルダを自動生成する Go/AWS CLI。README がほぼ空なので、対象 AWS リソースと実行引数をコードで確認する。 2024-04-02

graphvisualize B CSV グラフ、音声、動画を合成する Python ツール群。Python 3.9、Pipenv、matplotlib、OpenCV、MoviePy、ffmpeg が前提。各スクリプトの引数を確認して実行する。 2022-11-04

sandbox-account-manager B/C Vue 3 製の社内アカウント管理画面とみられる。npm install、npm run serve。README が雛形のため、接続先 API と運用状況の確認が必要。 2023-09-05

reactflow-test B/C Next.js、React Flow、Redux 系によるフローエディタ・実行 UI の活発な試作。npm install、npm run dev。名前は test だが実装量と 2025 年の更新は大きい。製品化方針を確認する。 2025-08-19

ws-sample-client B/C WebSocket クライアントのサンプル。2025 年に更新されたが README と標準マニフェストがない。対応サーバー、起動ファイル、通信仕様を確認する。 2025-11-28

ws-sample-server B/C Node.js と ws による WebSocket サンプルサーバー。npm install、npm start。ws-sample-client と対で利用する。 2025-11-26
eeg2influx C リポジトリ名は EEG データの InfluxDB 搬送を示すが、README と Go module が CLI テンプレートのまま。実装完成度を確認するまで業務利用しない。 2024-08-05
go-toyota-trigger C ・アーカイブ Go CLI テンプレートのままアーカイブ済み。名称上は案件用トリガーだが、README には実際の用途が残っていない。 2024-01-31
nocodejs-test C nocode-js の初期コミットのみ。技術検証用。README ・依存定義 ・実装目的が不足。 2025-06-11
azure-func-test C 空に近い Azure Functions 検証リポジトリ。更新履歴も残っていない。 -
azure-swa-vue-test C Vue 2、MSAL、Azure Static Web Apps の認証検証。npm install、npm run serve。現在の本番基盤ではなく技術検証として扱う。 2021-07-28
python-opencv-queue-multiprocessing C Python 3.10、OpenCV、NumPy による Queue/multiprocessing 検証。Pipenv で環境構築し、エントリースクリプトを確認する。 2023-06-27
python-osc-test C Python 3.10 と python-osc の通信検証。Pipenv で環境構築し、送受信スクリプトを個別に実行する。 2023-06-07
sandbox-recruit-tool C Vue 3、Quasar、Firebase の採用ツール。npm install、npm run serve。2023 年以降の運用状況を確認する。 2023-02-06
sandbox-experimental-tool C Vue 2、Vuetify、Firebase の実験管理系 Web ツール。npm install、npm run serve。2022 年以降更新がない。 2022-04-04
kashikaku-db-migration C Node.js と AWS SDK v3 による DynamoDB マイグレーション。README がいないため、対象テーブルと dry-run 可否を確認するまで実行しない。 2023-07-28
neume-landing-demo C Vue 2/Vuetify の Neume ランディングページ試作。npm install、npm run serve。README の名称にも不整合がある。 2021-07-02
eye-tracker-example C Python 3.9、SciPy、NumPy、OpenCV による視線マップ・相関解析例。Pipenv で環境構築し、個別スクリプトを実行する。 2021-05-17
tobii_eyetracker C Tobii 向け計測アプリ。Python 3.8、NumPy、pandas。README が短く、Tobii SDK や接続方法の記載が不足。 2022-03-17
aimlabtrigger C Go、RobotGo、キーボードフックを使う Aim Lab 用トリガーツール。OS 入力フックの権限と対応 OS を確認する。 2021-08-30
measureGUI C 古い計測 GUI。ログ、ノイズ判定、poor judge の履歴があるが、README と依存定義が不足。fg001 が後継が確認する。 2020-12-09
measuretool C MindWave を含む古い計測ツール。自動ポート選択などを実装。measureGUI／fg001 との後継関係を確認する。 2020-12-09
subjectanalyze C 被験者データ解析スクリプト。README がタイトルのみで、2020 年以降更新なし。eeganalyze への統合候補。 2020-12-07
tvtokyo-kddi-server C Go、Gin、WebSocket による案件用サーバー。README がいないため、起動前にエントリーポイントと設定を確認する。 2020 年以降更新なし。 2020-03-17
tvtokyo-kddi-visualizer C Nginx で配信する案件用リアルタイム可視化画面。Docker build/run 手順あり。上記サーバーと対で使う。 2020-03-17
realtime-saliency-mock C サリエンシー API を使ったリアルタイム表示モック。README ・依存定義がなく、2020 年以降更新なし。 2020-12-03
saliency-api C Flask API で画像／動画をヒートマップへ変換。Python 3.7 または Docker。/video2heat、/image2heat へ multipart POST する。 2021-06-11
saliency-web C サリエンシー Web アプリ。Python 3.9 の Flask/Docker 構成。docker build後、5000 番ポートで実行する構成。 2021-06-11
saliency-map C DeepGaze、CovSal などのサリエンシーモデル評価。Python 3.8、Pipenv、科学計算・Jupyter 系依存。研究検証用途。 2021-08-31

11 / 14

6. 機能別の使い分け

Web 実験を作る

* 実験本体: web-exp-tool

* 素材アップロード: web-exp-uploader

* 実験アカウント管理: exp-account-manager

* データ回収と Drive 転送: web-exp-db-manager

* 被験者 PC の操作制限: shortcutkey-disabler

デスクトップ実験を作る

* 刺激提示: fg002

* HuME 計測: fg001

* 複数 PC 時刻同期: time-sync-system

* 計測後解析: eeganalyze または treeflow

EEG 解析を追加する

* 再利用可能な低レベル処理: eeganalyze

* データフロー、遅延ロード、案件横断パイプライン: treeflow

* 案件別スクリプト: fieldstudy-script

* グラフと動画の合成: graphvisualize

サリエンシー解析を扱う

* AWS Lambda 群: saliency-functions

* 単純な Flask API: saliency-api

* Web UI: saliency-web

* モデル評価: saliency-map

* 古典的静止画処理: visual-saliency

* リアルタイム UI の旧モック: realtime-saliency-mock

AWS・社内システムを扱う

* イマジカ向け API/Lambda/DB: imagica-cloud-function

* 社内 AWS ネットワーク: milky-way

* Cognito アカウント作成: go-cognito-account-creator

* Neume アカウント作成: neume-account-creator

7. 運用上の共通注意

本番作業

本番環境へ変更を加える前に、対象環境、バックアップ、ロールバック手順、承認者を確認する。特に Firebase、Firestore、RDS、Cognito、WordPress、AWS ネットワークに対する操作は、ローカル実行であっても本番へ直接影響することがある。

認証情報

Firebase サービスアカウント、Google Drive 認証、AWS SSO、一時クレデンシャル、Cognito トークン、htpasswd は Git にコミットしない。README に具体的な個人名や過去の運用連絡先が含まれていても、新しい文書・ログへ転記しない。

開発環境

古いリポジトリでは、次の技術が混在している。

* Vue 2 / Vue 3

* Python 3.7～3.10

* Pipenv / Rye / pyproject

* Go 1.13～1.21

* 古い Nginx、Flask、AWS SDK、Firebase SDK

最新版へ一括更新せず、まず再現可能な既存環境を作り、テストと移行計画を用意する。

データ保護

実験ログ、EEG、視線、被験者回答、アカウント情報は個人情報・研究データとして扱う。ローカルコピー、Drive、S3、Firestore、RDS の保存場所とアクセス権を確認する。

既定ブランチ

develop が既定のリポジトリ:

- * sandbox-experimental-tool
- * realtime-saliency-mock
- * sandbox-corporate-website
- * web-exp-tool
- * neume-front-end

それ以外は主に master、azure-func-test のみ main である。

8. 整理・改善候補

優先して整備したいこと

1. time-sync-system に構成図、起動手順、ポート、ログ形式を追加する。
2. web-exp-tool の案件追加手順、環境変数、Firebase 環境の切替手順を文書化する。
3. fg001 と fg002 の配布版ビルド・実機確認手順を CI／チェックリスト化する。
4. eeganalyze、treeflow、fieldstudy-script の責務境界を明文化する。
5. imagica-cloud-function のデプロイ順と DB マイグレーションのロールバック手順を追加する。

利用状況を確認して整理するもの

次は、実際の利用者がいないことを確認したうえで、アーカイブするか、README へ「検証用」「保守終了」などを明記する候補である。

- * azure-func-test
- * nocodejs-test

12 / 14

- * python-osc-test
- * python-opencv-queue-multiprocessing
- * azure-swa-vue-test
- * realtime-saliency-mock
- * neume-landing-demo
- * tvtokyo-kddi-server
- * tvtokyo-kddi-visualizer
- * microstate

後継関係を整理するもの

- * measuretool / measureGUI → fg001
- * subjectanalyze / 案件別 eeganalyze スクリプト → fieldstudy-script または treeflow
- * saliency-api / saliency-web / realtime-saliency-mock → saliency-functions
- * eeg2influx → 完成しているか、別実装へ移行したか
- * reactflow-test → 製品化するか、純粋な検証として残すか

9. 全リポジトリ索引

1. eeganalyze
2. web-exp-uploader
3. azure-func-test
4. go-toyota-trigger (アーカイブ済み)
5. sandbox-experimental-tool
6. microstate
7. neume-account-creator
8. eye-tracker-example
9. python-opencv-queue-multiprocessing
10. realtime-saliency-mock
11. sandbox-corporate-proxy

12. shortcutkey-disabler
13. tobii_eyetracker
14. saliency-api
15. ws-sample-client
16. tvtokyo-kddi-visualizer
17. trigger-splitter
18. sandbox-account-manager
19. milky-way
20. ws-sample-server
21. time-sync-system
22. neume-landing-demo
23. sandbox-corporate-website
24. web-exp-tool
25. kashikaku-db-migration
26. saliency-map
27. fg001
28. nocodejs-test
29. sandbox-recruit-tool
30. python-osc-test
31. reactflow-test
32. graphvisualize
33. fg002
34. saliency-web
35. subjectanalyze
36. visual-saliency
37. neume-front-end
38. saliency-functions
39. aimlabtrigger
40. measureGUI
41. tvtokyo-kddi-server
42. web-exp-db-manager
43. fieldstudy-script
44. go-cognito-account-creator
45. azure-swa-vue-test
46. treeflow
47. exp-account-manager
48. eeg2influx
49. measuretool
50. imagica-cloud-function
13 / 14

10. 記載範囲

- * 内容は既定ブランチを中心に整理している。
- * 既定ブランチ以外の最新コミット、未マージ PR、GitHub Actions、外部 Notion の本文までは追っていない。
- * README がないリポジトリは、実行手順を完全には確定できない。
- * 更新が古くても、顧客契約、法令、研究再現性、データ保持の都合で削除できない場合がある。
- * 「C」は削除推奨ではなく、現時点で日常運用の優先度が低い、または追加確認が必要という分類である。

14 / 14